- Graph V2: Multi-Agent LangGraph Architecture
  - Table of Contents
  - Overview
    - Key Design Principles
  - Architecture Diagram
  - Core Concepts
    - 1. LangGraph Fundamentals
    - 2. Multi-Agent Pattern
    - 3. Manager QA Pattern
    - 4. Tool Execution Pattern
  - State Management
    - AnalysisState Structure
    - State Updates
    - Message Types
  - Graph Structure
    - Stages and Nodes
    - Graph Assembly
  - Specialist Agents
    - System Prompts
      - Data Loader Prompt
      - Analyst Prompt
      - Code Reviewer Prompt
      - Reporter Prompt
      - Manager Review Prompt
    - Specialist Node Factory
  - Manager Review System
    - Manager Review Node
    - Manager Router
  - Tool Execution
    - Tool Node
    - Specialist Router
    - After Tools Router
  - Context Building
    - Focused Context Builder
    - Message Compression
    - Tool Message Stripping
  - Message Flow

    - Complete Message Sequence Example
- Routing Logic
    - Edge Types
    - Routing Functions
        - Specialist Router
        - After Tools Router
        - Manager Router
- Error Handling
    - Specialist Node Errors
    - Tool Node Errors
    - Manager Review Errors
    - Iteration Caps
- Token Budget Controls
    - Constants
    - Truncation Function
    - Application Points
- Complete Execution Flow
    - Step-by-Step Pipeline
    - Rework Scenario
- Extension Guide
    - Adding a New Specialist
        - Step 1: Define System Prompt
        - Step 2: Add to STAGES
        - Step 3: Define Tool Set
        - Step 4: Add to PROMPTS
        - Step 5: Update Manager Review (if needed)
    - Adding New Routing Logic
        - Example: Conditional Specialist Selection
    - Modifying Context Building
        - Example: Add More Context for Specific Stage
    - Adding Error Recovery
        - Example: Retry Failed Tool Calls
- Summary

Graph V2: Multi-Agent LangGraph Architecture

Table of Contents

- Overview
- Architecture Diagram
- Core Concepts
- State Management
- Graph Structure
- Specialist Agents
- Manager Review System
- Tool Execution
- Context Building
- Message Flow
- Routing Logic
- Error Handling
- Token Budget Controls
- Complete Execution Flow
- Extension Guide

Overview

Graph V2 implements a **deterministic multi-agent workflow** for ore dressing plant analysis. The system uses a hybrid approach:

- **Deterministic stage order**: Agents work in a fixed sequence (data_loader → analyst → code_reviewer → reporter)
- **Manager QA review**: After each stage, a manager reviews quality and can request rework
- **Tool-based execution**: Agents use MCP tools to load data, execute analysis, and generate reports
- **Focused context**: Each agent receives only relevant information to stay within token limits

Key Design Principles

1. **Separation of Concerns**: Each specialist has a single, well-defined responsibility

2. **Quality Gates**: Manager review ensures high-quality output before proceeding
3. **Token Efficiency**: Context compression prevents exceeding LLM limits
4. **Deterministic Flow**: Predictable stage progression enables reliable debugging
5. **Rework Capability**: Manager can send agents back for improvements (1 rework per stage)

Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              USER REQUEST                                 │
│                      "Analyze Mill 8 for 30 days"                         │
└──────────────────────────────────────────────────────────────────────────┘
                                   │
                                   │
                                   ▼

┌──────────────────────────────────────────────────────────────────────────┐
│                              START                                        │
│                              ↓                                            │
│                      data_loader_entry                                    │
│                              ↓                                            │
│                        data_loader                                        │
│                        ↓ (has tool_calls?)                                │
│                      ┌──────┴──────┐                                      │
│                      │             │                                      │
│                YES   │             │ NO                                   │
│                      ↓             ↓                                      │
│                    tools      manager_review                              │
│                      │             │                                      │
│                      │             ↓ (ACCEPT?)                            │
│                      │         ┌───┴───┐                                  │
│                      │         │       │                                  │
│                      │      ACCEPT    REWORK                              │
│                      │         │       │                                  │
│                      │         ↓       ↓                                  │
│                      │   analyst_entry  data_loader_entry                 │
│                      │         │       │                                  │
│                      └─────────┴───────┘                                  │
│                              ↓                                            │
│                        analyst                                            │
│                        ↓ (has tool_calls?)                                │
│                      ┌──────┴──────┐                                      │
│                      │             │                                      │
│                YES   │             │ NO                                   │
│                      ↓             ↓                                      │
│                    tools      manager_review                              │
│                      │             │                                      │
│                      │             ↓ (ACCEPT?)                            │
│                      │         ┌───┴───┐                                  │
│                      │         │       │                                  │
│                      │      ACCEPT    REWORK                              │
```

```
│                    │        │         │                                    │
│                    │        ↓         ↓                                     │
│                    │  code_reviewer_entry  analyst_entry                   │
│                    │        │         │                                     │
│                    └────────┴─────────┘                                     │
│                           ↓                                                 │
│                      code_reviewer                                          │
│                      ↓ (has tool_calls?)                                    │
│                    ┌────────┴────────┐                                      │
│                    │                 │                                      │
│              YES   │                 │ NO                                   │
│                    ↓                 ↓                                      │
│                 tools      manager_review                                   │
│                    │                 │                                      │
│                    │                 ↓ (ACCEPT?)                            │
│                    │            ┌────┴────┐                                 │
│                    │            │         │                                 │
│                    │         ACCEPT    REWORK                               │
│                    │            │         │                                 │
│                    │            ↓         ↓                                 │
│                    │   reporter_entry  code_reviewer_entry                  │
│                    │        │         │                                     │
│                    └────────┴─────────┘                                     │
│                           ↓                                                 │
│                      reporter                                               │
│                      ↓ (has tool_calls?)                                    │
│                    ┌────────┴────────┐                                      │
│                    │                 │                                      │
│              YES   │                 │ NO                                   │
│                    ↓                 ↓                                      │
│                 tools      manager_review                                   │
│                    │                 │                                      │
│                    │                 ↓ (ACCEPT?)                            │
│                    │            ┌────┴────┐                                 │
│                    │            │         │                                 │
│                    │         ACCEPT    REWORK                               │
│                    │            │         │                                 │
│                    │            ↓         ↓                                 │
│                    │           END    reporter_entry                        │
│                    │                                                        │
└────────────────────────────────────────────────────────────────────────────┘
```

Core Concepts

1. LangGraph Fundamentals

LangGraph is a framework for building **stateful, multi-actor applications with LLMs**. Key concepts:

State: Shared data structure that persists across nodes

```
class AnalysisState(MessagesState):
    current_stage: str          # Which stage is currently active
    stage_attempts: dict       # How many times each stage has been attempted
```

Nodes: Functions that process state and return updates

```
def specialist_node(state: AnalysisState) -> dict:
    # Process state, call LLM, return message
    return {"messages": [response]}
```

Edges: Define how control flows between nodes

```
graph.add_edge("node_a", "node_b")  # Always go from A to B
graph.add_conditional_edges(         # Conditional routing
    "node_a",
    router_function,
    {"option1": "node_b", "option2": "node_c"}
)
```

2. Multi-Agent Pattern

The graph uses a **specialist agent pattern** where each agent has:

- **Specific role**: One clear responsibility (data loading, analysis, review, reporting)
- **Tool access**: Only tools relevant to their role
- **System prompt**: Instructions defining their behavior
- **Iteration limits**: Cap on how many times they can loop

Why Specialists?

- **Focus**: Each agent specializes in one task
- **Parallelism**: Different agents can work independently
- **Testability**: Each agent can be tested in isolation
- **Scalability**: Easy to add new specialists

3. Manager QA Pattern

The manager acts as a **quality gate** between stages:

```
specialist → manager_review → (ACCEPT → next stage) OR (REWORK → same specialist)
```

Manager Responsibilities:

- Review specialist output for completeness
- Check quality (charts, analysis depth)
- Validate correctness (no errors, reasonable values)
- Decide: ACCEPT or REWORK with specific feedback

Rework Limits:

- Maximum 1 rework per stage
- Prevents infinite loops
- Forces progression after reasonable effort

4. Tool Execution Pattern

Agents use tools through a **tool loop**:

```
specialist → (makes tool_calls) → tools → (returns results) → specialist
```

Tool Loop Example:

```
1. Specialist: "I need to load data for Mill 8"
   → Calls query_mill_data(mill_number=8)

2. Tool Node: Executes query, returns data

3. Specialist: Receives data, decides next action
   → "Data loaded, now I'll analyze it"
   → Calls execute_python(code="...")

4. Tool Node: Executes Python, returns results

5. Specialist: "Analysis complete, no more tools needed"
   → No tool_calls → goes to manager_review
```

State Management

AnalysisState Structure

```
class AnalysisState(MessagesState):
    current_stage: str          # e.g., "data_loader", "analyst"
    stage_attempts: dict         # {"data_loader": 1, "analyst": 2, ...}
```

Inherits from MessagesState:

```
class MessagesState(TypedDict):
    messages: list[BaseMessage]  # All conversation history
```

State Updates

Each node returns a dict with state updates:

```
def specialist_node(state: AnalysisState) -> dict:
    # ... processing ...
    return {
        "messages": [AIMessage(content="...", name="data_loader")],
        # "current_stage": "analyst"  # Updated by entry nodes
        # "stage_attempts": {...}    # Updated by manager
    }
```

Key Patterns:

- **Append-only messages**: Never remove messages, only add new ones
- **Stage tracking**: Entry nodes update `current_stage`
- **Attempt counting**: Manager increments `stage_attempts` on each review

Message Types

```
HumanMessage    # User input
SystemMessage   # Agent instructions
```

```
AIMessage       # LLM responses (may include tool_calls)
ToolMessage     # Tool execution results
```

Message Flow:

```
HumanMessage → AIMessage (with tool_calls) → ToolMessage → AIMessage → ...
```

Graph Structure

Stages and Nodes

```
STAGES = ["data_loader", "analyst", "code_reviewer", "reporter"]
MAX_REWORKS_PER_STAGE = 1
```

Node Types:

1. **Entry Nodes** (4 total): Set the current stage

   ```
   data_loader_entry → sets current_stage = "data_loader"
   analyst_entry → sets current_stage = "analyst"
   code_reviewer_entry → sets current_stage = "code_reviewer"
   reporter_entry → sets current_stage = "reporter"
   ```

2. **Specialist Nodes** (4 total): Execute agent logic

   ```
   data_loader → Loads data using query_mill_data
   analyst → Performs analysis using execute_python
   code_reviewer → Validates outputs
   reporter → Writes markdown report
   ```

3. **Tool Node** (1): Executes tool calls

   ```
   tools → Dispatches to MCP tools, returns results
   ```

4. **Manager Review Node** (1): Quality gate

```
manager_review → Decides ACCEPT or REWORK
```

Graph Assembly

```
graph = StateGraph(AnalysisState)

# Add all nodes
for stage in STAGES:
    graph.add_node(f"{stage}_entry", make_stage_entry(stage))
    graph.add_node(stage, make_specialist_node(stage))

graph.add_node("tools", tool_node)
graph.add_node("manager_review", manager_review_node)

# Set entry point
graph.set_entry_point("data_loader_entry")

# Wire edges
for stage in STAGES:
    graph.add_edge(f"{stage}_entry", stage)
    graph.add_conditional_edges(stage, specialist_router, {
        "tools": "tools",
        "manager_review": "manager_review"
    })
```

Specialist Agents

System Prompts

Each specialist has a detailed system prompt defining their role:

Data Loader Prompt

```
DATA_LOADER_PROMPT = f"""{DOMAIN_CONTEXT}

You are the Data Loader. Your ONLY job is to call query_mill_data to load data.

CRITICAL: ALWAYS compute start_date and end_date to filter at SQL level. Never load
```

```
full tables.
- "last 24 hours" → end_date = today's date, start_date = yesterday's date (ISO
format YYYY-MM-DD)
- "last 30 days" → end_date = today, start_date = 30 days ago
- "last week" → end_date = today, start_date = 7 days ago
- Today's date: {{TODAY_DATE}}. Use this to compute date ranges.
- If no time range is mentioned, default to last 30 days.

RULES:
- Extract mill number(s) from the request. If "all mills" → load mills 1 through
12.
- ALWAYS pass start_date and end_date to every query_mill_data call.
- Each mill is stored automatically as 'mill_data_N' (e.g. mill_data_1,
mill_data_8).
- After loading, write a brief summary: mills loaded, rows per mill, date range.
- Do NOT call any other tool. Do NOT analyze the data."""
```

Key Points:

- **Single responsibility**: ONLY load data
- **Date computation**: Must calculate dates from natural language
- **Efficiency**: Always filter at SQL level (don't load full tables)
- **No analysis**: Stop after loading, don't try to analyze

Analyst Prompt

```
ANALYST_PROMPT = f"""{DOMAIN_CONTEXT}

You are the Data Analyst. Data has been loaded by the data_loader.

ACCESSING DATA:
- Call list_dfs() first to see all loaded DataFrames and their shapes
- Single mill: df = get_df('mill_data_8')  — for mill 8
- Multiple mills: use a loop: for i in range(1, 13): df =
get_df(f'mill_data_{{i}}')
- The variable `df` is pre-set to the first loaded DataFrame

Call execute_python with a SINGLE block of Python code.
ALWAYS start your code with:
```

print("Loaded DataFrames:", list_dfs())

```
For MULTI-MILL comparison requests:
- Build a summary dict/DataFrame with stats per mill (mean Ore, std, etc.)
- Create bar charts comparing mills side-by-side
- Use the LAST 24 hours of data if requested: df_recent = df[df.index >=
df.index.max() - pd.Timedelta(hours=24)]
```

```
For SINGLE-MILL analysis requests:
- EDA: distributions for Ore, PSI80, DensityHC, MotorAmp
- SPC: control charts (mean ± 3σ) for PSI80 and Ore
- Correlations: heatmap of df.corr()
- Anomaly detection: Z-scores (threshold=3)
- Downtime: periods where Ore < 10 t/h

CHART QUALITY RULES (CRITICAL):
- Use `sns.set_theme(style='whitegrid', font_scale=1.2)` at the start
- Figure sizes: bar charts (14,7), distributions (10,6), SPC (14,5), heatmap
(12,10)
- All axes MUST have labels with units: 'Ore Feed Rate (t/h)', 'PSI80 (µm)', etc.
- All charts MUST have descriptive titles
- For bar charts: use different colors per mill, add value labels on bars, rotate
x-labels if needed
- Save ALL charts: plt.savefig(os.path.join(OUTPUT_DIR, 'filename.png'), dpi=150,
bbox_inches='tight')
- ALWAYS call plt.close() after each savefig

OUTPUT RULES:
- Print all key statistics to stdout (the reporter will use these numbers)
- Print a summary table at the end"""
```

Key Points:

- **Data access**: Use `get_df()` and `list_dfs()`
- **Code structure**: Single execute_python call with full analysis
- **Chart quality**: Specific formatting rules for professional output
- **Output**: Print statistics for reporter to use

Code Reviewer Prompt

```
CODE_REVIEWER_PROMPT = f"""{DOMAIN_CONTEXT}

You are the Code Reviewer. Validate the analysis outputs.

1. Call list_output_files to see what charts were generated
2. Review the stdout from previous steps for errors
3. If there are NO chart files (.png), call execute_python to regenerate them
4. If charts exist and stdout has meaningful results, write a short validation
summary

Do NOT regenerate charts that already exist. Only fix errors or fill gaps."""
```

Key Points:

- **Validation focus**: Check for missing charts or errors

- **Minimal intervention**: Only regenerate if necessary
- **Summary output**: Provide brief validation report

Reporter Prompt

```
REPORTER_PROMPT = f"""{DOMAIN_CONTEXT}

You are the Reporter. Write a COMPREHENSIVE professional Markdown analysis report.

STEPS:
1. Call list_output_files to get the exact chart filenames
2. Read through ALL previous messages to extract statistics and findings
3. Call write_markdown_report with the full report content

Report MUST include:
- **Title** matching the analysis request
- **Executive Summary**: 3-4 sentences with the key findings and actual numbers
- **Data Overview**: what data was loaded, time range, number of records
- **Findings sections**: organize by the analyses performed, include actual numbers
from stdout
- **Charts**: embed EVERY .png file from list_output_files using ![description]
(exact_filename.png)
- **Conclusions & Recommendations**: 3-5 specific actionable items

CRITICAL RULES:
- Use EXACT filenames from list_output_files — only reference charts that actually
exist
- Include ACTUAL numbers from the analysis (means, stds, counts) — extract from
prior messages
- Do NOT use placeholder text — use real numbers from the analysis
- Every section must have substantive content, not just headers
- The report should be at least 1000 words"""
```

Key Points:

- **Comprehensive**: Include all sections with actual data
- **Real data**: No placeholders, use actual numbers from analysis
- **Professional**: Plant manager-ready format

Manager Review Prompt

```
MANAGER_REVIEW_PROMPT = f"""{DOMAIN_CONTEXT}

You are the Quality Manager reviewing the output of a specialist agent.
Your job is to decide if the work is ACCEPTABLE or needs REWORK.

Evaluate the last specialist's output for:
- Completeness: did they do everything asked?
```

```
- Quality: are charts well-formatted? Is the report detailed with actual numbers?
- Correctness: are there errors or unreasonable values?

Respond with EXACTLY one of:
- ACCEPT: [brief reason] — if the work is good enough to proceed
- REWORK: [specific instructions on what to fix] — if improvements are needed

Be concise. One line is enough."""
```

Key Points:

- **Binary decision**: ACCEPT or REWORK
- **Specific feedback**: Clear instructions for rework
- **Quality criteria**: Completeness, quality, correctness

Specialist Node Factory

```python
def make_specialist_node(name: str):
    system_prompt = PROMPTS[name]
    stage_llm = specialist_llms[name]  # LLM with tools bound

    def specialist_node(state: AnalysisState) -> dict:
        iteration = sum(1 for m in state["messages"]
                        if getattr(m, "name", None) == name) + 1

        if iteration > MAX_SPECIALIST_ITERS:
            return {
                "messages": [AIMessage(
                    content=f"[{name}] Done (iteration cap). Moving on.",
                    name=name,
                )],
            }

        # Build focused context
        focused = build_focused_context(state["messages"], name)
        messages = [SystemMessage(content=system_prompt)] + focused

        # Call LLM
        response = stage_llm.invoke(messages)
        response.name = name

        return {"messages": [response]}

    return specialist_node
```

Key Features:

- **Iteration tracking**: Count how many times specialist has run

- **Iteration cap**: Prevents infinite loops (MAX_SPECIALIST_ITERS = 5)
- **Focused context**: Only relevant messages, not entire history
- **Tool binding**: Each specialist has their own tool set

Manager Review System

Manager Review Node

```
def manager_review_node(state: AnalysisState) -> dict:
    current = state.get("current_stage", "data_loader")
    attempts = state.get("stage_attempts", {})
    attempt_count = attempts.get(current, 0)

    # Skip review for data_loader (just load and move on)
    if current == "data_loader":
        return {
            "messages": [AIMessage(content="ACCEPT: Data loaded successfully.",
                                   name="manager")],
            "stage_attempts": {**attempts, current: attempt_count + 1},
        }

    # If already reworked max times, force accept
    if attempt_count >= MAX_REWORKS_PER_STAGE:
        return {
            "messages": [AIMessage(
                content=f"ACCEPT: Max reworks reached for {current}.",
                name="manager"
            )],
            "stage_attempts": {**attempts, current: attempt_count + 1},
        }

    # Compress messages and strip tool calls
    compressed = compress_messages(state["messages"])
    messages = [SystemMessage(content=MANAGER_REVIEW_PROMPT)] + \
               strip_tool_messages(compressed)

    # Call LLM for review
    response = llm.invoke(messages)
    content = str(response.content).strip()

    # Parse decision
    if content.upper().startswith("REWORK"):
        decision = "REWORK"
    else:
        decision = "ACCEPT"

    stamped = f"{decision}: {content}"
```

```
    return {
        "messages": [AIMessage(content=stamped, name="manager")],
        "stage_attempts": {**attempts, current: attempt_count + 1},
    }
```

Key Logic:

1. **Skip data_loader**: No quality check needed, just accept
2. **Rework limit**: Force accept after MAX_REWORKS_PER_STAGE attempts
3. **Message preparation**: Compress and strip tool messages for LLM
4. **Decision parsing**: Check if response starts with "REWORK"

Manager Router

```
def manager_router(state: AnalysisState) -> str:
    """After manager review: advance to next stage or rework current."""
    last = state["messages"][-1]
    content = last.content if isinstance(last.content, str) else str(last.content)
    current = state.get("current_stage", "data_loader")

    # Only rework if message explicitly starts with REWORK:
    if content.startswith("REWORK:"):
        return f"{current}_entry"

    # Everything else (ACCEPT:, errors, etc.) → advance
    idx = STAGES.index(current) if current in STAGES else 0
    if idx + 1 < len(STAGES):
        next_stage = STAGES[idx + 1]
        return f"{next_stage}_entry"

    return "end"
```

Routing Logic:

```
Manager Decision → REWORK → current_stage_entry (rework same stage)
                 → ACCEPT → next_stage_entry (advance)
                 → END (if last stage)
```

Tool Execution

Tool Node

```python
async def tool_node(state: AnalysisState) -> dict:
    last_message = state["messages"][-1]
    results = []

    for tc in last_message.tool_calls:
        tool = tools_by_name.get(tc["name"])
        if tool is None:
            results.append(ToolMessage(
                content=f"Error: unknown tool '{tc['name']}'",
                tool_call_id=tc["id"],
                name=tc["name"],
            ))
            continue

        try:
            print(f"    [tool] Executing {tc['name']}...")
            output = await tool.ainvoke(tc["args"])
            results.append(ToolMessage(
                content=str(output),
                tool_call_id=tc["id"],
                name=tc["name"],
            ))
        except Exception as e:
            results.append(ToolMessage(
                content=f"Error: {e}",
                tool_call_id=tc["id"],
                name=tc["name"],
            ))

    return {"messages": results}
```

Key Features:

1. **Extract tool_calls**: From last AIMessage
2. **Execute tools**: Call `tool.ainvoke(arguments)`
3. **Handle errors**: Return error messages as ToolMessage
4. **Return results**: All results as ToolMessage list

Specialist Router

```python
def specialist_router(state: AnalysisState) -> str:
    """After specialist: go to tools if tool_calls, else to manager review."""
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
```

```
        return "tools"
    return "manager_review"
```

Routing Logic:

```
Specialist Response → has tool_calls? → YES → tools
                                        → NO → manager_review
```

After Tools Router

```
def after_tools(state: AnalysisState) -> str:
    """After tool execution, return to the specialist that called them."""
    for msg in reversed(state["messages"]):
        if hasattr(msg, "tool_calls") and msg.tool_calls and \
           getattr(msg, "name", None):
            return msg.name
    return "data_loader"
```

Routing Logic:

```
Tool Results → Find which specialist called them → Return to that specialist
```

Why reversed?

- Most recent specialist is at the end of messages
- Search backwards to find the last specialist with tool_calls

---

Context Building

Focused Context Builder

```
def build_focused_context(all_msgs: list[BaseMessage], stage_name: str) ->
list[BaseMessage]:
    """Give each specialist ONLY what it needs:
    1. The original user request (HumanMessage)
    2. A compact summary of prior stages (data loaded, analysis results)
```

```
    3. This stage's own messages + tool results (for the tool loop)
    """
    user_msg = None
    prior_summary_parts = []
    current_stage_msgs = []

    # Identify which tool_call_ids belong to this stage
    my_tool_call_ids = set()
    for msg in all_msgs:
        if isinstance(msg, AIMessage) and \
           getattr(msg, "name", None) == stage_name and \
           msg.tool_calls:
            for tc in msg.tool_calls:
                my_tool_call_ids.add(tc.get("id"))

    for msg in all_msgs:
        # 1. Capture original user request
        if isinstance(msg, HumanMessage) and user_msg is None:
            user_msg = msg
            continue

        msg_name = getattr(msg, "name", None)

        # 2. Messages from THIS stage → keep them for the tool loop
        if msg_name == stage_name:
            current_stage_msgs.append(msg)
            continue

        # Tool results for THIS stage's tool calls → keep
        if isinstance(msg, ToolMessage) and msg.tool_call_id in my_tool_call_ids:
            content = normalize_content(msg.content)
            current_stage_msgs.append(ToolMessage(
                content=truncate(content, MAX_TOOL_OUTPUT_CHARS),
                tool_call_id=msg.tool_call_id,
                name=msg.name,
            ))
            continue

        # 3. Messages from OTHER stages → summarize compactly
        if isinstance(msg, ToolMessage) and msg.name == "query_mill_data":
            content = normalize_content(msg.content)
            if "loaded" in content.lower():
                prior_summary_parts.append(truncate(content, 150))
        elif isinstance(msg, ToolMessage) and msg.name == "execute_python":
            content = normalize_content(msg.content)
            prior_summary_parts.append(f"[python output]: {truncate(content,
800)}")
        elif isinstance(msg, AIMessage) and msg_name and \
             msg_name != "manager" and not msg.tool_calls:
            content = normalize_content(msg.content)
            if content:
                prior_summary_parts.append(f"[{msg_name}]: {truncate(content,
300)}")
        elif msg_name == "manager" and isinstance(msg, AIMessage):
            content = normalize_content(msg.content)
            if "REWORK" in content and stage_name in content:
                current_stage_msgs.append(msg)  # manager feedback for this stage
```

```
    # Assemble
    result = []
    if user_msg:
        result.append(user_msg)

    if prior_summary_parts:
        # Compact: only last few summaries to keep context tight
        summary = "[Prior analysis context]:\n" + \
                 "\n".join(prior_summary_parts[-6:])
        result.append(HumanMessage(content=summary))

    result.extend(compress_messages(current_stage_msgs))
    return result
```

Key Strategies:

1. **User request**: Always include original HumanMessage
2. **Prior stage summaries**: Compact summaries of other stages
3. **Current stage full**: Full messages for current stage (tool loop needs this)
4. **Manager feedback**: Include if REWORK for this stage

Why This Matters:

- **Token efficiency**: Don't send entire message history to each agent
- **Focus**: Each agent sees only what's relevant to their task
- **Tool loop preservation**: Current stage needs full context for multi-turn tool calls

Message Compression

```
def compress_messages(messages: list[BaseMessage]) -> list[BaseMessage]:
    if len(messages) > MAX_MESSAGES_WINDOW + 1:
        messages = [messages[0]] + messages[-(MAX_MESSAGES_WINDOW):]

    compressed = []
    for msg in messages:
        if isinstance(msg, ToolMessage):
            content = normalize_content(msg.content)
            compressed.append(ToolMessage(
                content=truncate(content, MAX_TOOL_OUTPUT_CHARS),
                tool_call_id=msg.tool_call_id,
                name=msg.name,
            ))
        elif isinstance(msg, AIMessage):
            content = normalize_content(msg.content)
            if len(content) > MAX_AI_MSG_CHARS:
                compressed.append(AIMessage(
                    content=truncate(content, MAX_AI_MSG_CHARS),
```

```
                    name=getattr(msg, "name", None),
                    tool_calls=msg.tool_calls if msg.tool_calls else [],
                ))
            else:
                compressed.append(msg)
        else:
            compressed.append(msg)
    return compressed
```

Compression Rules:

1. **Window limit**: Keep only last MAX_MESSAGES_WINDOW messages
2. **Tool output**: Truncate to MAX_TOOL_OUTPUT_CHARS (2000)
3. **AI messages**: Truncate to MAX_AI_MSG_CHARS (3000)
4. **Preserve structure**: Keep message types and metadata

Tool Message Stripping

```
def strip_tool_messages(messages: list[BaseMessage]) -> list[BaseMessage]:
    """Remove ToolMessages and tool_calls for the manager (no tools bound)."""
    clean = []
    for msg in messages:
        if isinstance(msg, ToolMessage):
            content = normalize_content(msg.content)
            clean.append(AIMessage(
                content=f"[Tool result from {msg.name}]: {truncate(content, 800)}",
                name=msg.name,
            ))
        elif isinstance(msg, AIMessage) and msg.tool_calls:
            text = normalize_content(msg.content) or \
                  f"[{getattr(msg, 'name', 'agent')} requested tools]"
            clean.append(AIMessage(content=text, name=getattr(msg, "name", None)))
        else:
            clean.append(msg)
    return clean
```

Why Strip for Manager?

- Manager doesn't have tools bound
- ToolMessages would confuse the manager
- Convert to AIMessage summaries for readability

Message Flow

Complete Message Sequence Example

```
1. HumanMessage: "Analyze Mill 8 for the last 30 days"

2. [data_loader_entry] → current_stage = "data_loader"

3. [data_loader] AIMessage:
   "I need to load data for Mill 8 from 30 days ago to today."
   tool_calls: [
     {"name": "query_mill_data", "arguments": {"mill_number": 8, "start_date":
"2025-02-12", ...}}
   ]

4. [tools] ToolMessage:
   {"status": "loaded", "rows": 43200, "date_range": {...}}

5. [data_loader] AIMessage:
   "Data loaded successfully. Mill 8 data: 43200 rows from 2025-02-12 to 2025-03-
13."

6. [manager_review] AIMessage:
   "ACCEPT: Data loaded successfully."

7. [analyst_entry] → current_stage = "analyst"

8. [analyst] AIMessage:
   "I'll analyze the data. First, let me see what's available."
   tool_calls: [
     {"name": "list_dfs", "arguments": {}}
   ]

9. [tools] ToolMessage:
   {"dataframes": {"mill_data_8": (43200, 14)}}

10. [analyst] AIMessage:
    "I'll perform EDA and create charts."
    tool_calls: [
      {"name": "execute_python", "arguments": {"code": "..."}}
    }

11. [tools] ToolMessage:
    {"stdout": "Ore mean: 150.2, PSI80 mean: 23.5...", "new_files":
["ore_psi80_trend.png"]}

12. [analyst] AIMessage:
    "Analysis complete. Generated 1 chart with key statistics."

13. [manager_review] AIMessage:
    "ACCEPT: Charts generated with proper formatting and statistics."

14. [code_reviewer_entry] → current_stage = "code_reviewer"

15. [code_reviewer] AIMessage:
    "Checking outputs..."
```

```
    tool_calls: [
      {"name": "list_output_files", "arguments": {}}
    ]

16. [tools] ToolMessage:
    {"files": [{"name": "ore_psi80_trend.png", "size_kb": 45.2}]}

17. [code_reviewer] AIMessage:
    "Validation complete. 1 chart found, no errors detected."

18. [manager_review] AIMessage:
    "ACCEPT: Validation passed."

19. [reporter_entry] → current_stage = "reporter"

20. [reporter] AIMessage:
    "I'll create the report."
    tool_calls: [
      {"name": "list_output_files", "arguments": {}}
    ]

21. [tools] ToolMessage:
    {"files": [{"name": "ore_psi80_trend.png", "size_kb": 45.2}]}

22. [reporter] AIMessage:
    "Writing report with findings..."
    tool_calls: [
      {"name": "write_markdown_report", "arguments": {"filename":
"mill_8_analysis.md", "content": "..."}}
    ]

23. [tools] ToolMessage:
    {"status": "written", "file": "mill_8_analysis.md", ...}

24. [reporter] AIMessage:
    "Report written successfully."

25. [manager_review] AIMessage:
    "ACCEPT: Report complete."

26. [manager_router] → END

Final state["messages"][-1] contains the reporter's final output.
```

Routing Logic

Edge Types

1. Direct Edges: Always go from A to B

```
graph.add_edge("data_loader_entry", "data_loader")
```

2. Conditional Edges: Route based on function result

```
graph.add_conditional_edges(
    "data_loader",
    specialist_router,  # Function that returns string
    {"tools": "tools", "manager_review": "manager_review"}
)
```

Routing Functions

Specialist Router

```
def specialist_router(state: AnalysisState) -> str:
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
        return "tools"
    return "manager_review"
```

Decision Tree:

```
Last message has tool_calls?
  ├─ YES → "tools" (execute tools)
  └─ NO → "manager_review" (review quality)
```

After Tools Router

```
def after_tools(state: AnalysisState) -> str:
    for msg in reversed(state["messages"]):
        if hasattr(msg, "tool_calls") and msg.tool_calls and \
           getattr(msg, "name", None):
            return msg.name
    return "data_loader"
```

Decision Tree:

```
Find last specialist with tool_calls
  ├─ Found → return specialist name (e.g., "analyst")
  └─ Not found → return "data_loader" (fallback)
```

Manager Router

```python
def manager_router(state: AnalysisState) -> str:
    last = state["messages"][-1]
    content = str(last.content).strip()
    current = state.get("current_stage", "data_loader")

    if content.startswith("REWORK:"):
        return f"{current}_entry"  # Rework same stage

    idx = STAGES.index(current) if current in STAGES else 0
    if idx + 1 < len(STAGES):
        next_stage = STAGES[idx + 1]
        return f"{next_stage}_entry"  # Advance to next stage

    return "end"  # Pipeline complete
```

Decision Tree:

```
Manager decision starts with "REWORK:"?
  ├─ YES → current_stage_entry (rework)
  └─ NO → Is there a next stage?
          ├─ YES → next_stage_entry (advance)
          └─ NO → "end" (complete)
```

Error Handling

Specialist Node Errors

```python
def specialist_node(state: AnalysisState) -> dict:
    try:
        response = stage_llm.invoke(messages)
    except Exception as e:
        error_str = str(e)
        print(f"  [{name}] LLM error: {error_str[:200]}")
        return {
```

```
            "messages": [AIMessage(
                content=f"[{name}] Error: {error_str[:150]}. Moving on.",
                name=name,
            )],
        }
```

Error Handling:

- Catch LLM invocation errors
- Log error message
- Return error AIMessage
- Continue to next stage (don't block pipeline)

Tool Node Errors

```
async def tool_node(state: AnalysisState) -> dict:
    for tc in last_message.tool_calls:
        tool = tools_by_name.get(tc["name"])
        if tool is None:
            results.append(ToolMessage(
                content=f"Error: unknown tool '{tc['name']}'",
                tool_call_id=tc["id"],
                name=tc["name"],
            ))
            continue

        try:
            output = await tool.ainvoke(tc["args"])
            results.append(ToolMessage(content=str(output), ...))
        except Exception as e:
            results.append(ToolMessage(
                content=f"Error: {e}",
                tool_call_id=tc["id"],
                name=tc["name"],
            ))
```

Error Handling:

- Unknown tool → Return error message
- Tool execution error → Return error message
- Specialist sees error and can retry or adjust

Manager Review Errors

```python
def manager_review_node(state: AnalysisState) -> dict:
    try:
        response = llm.invoke(messages)
    except Exception as e:
        print(f"  [manager] Review error: {str(e)[:150]}")
        return {
            "messages": [AIMessage(
                content="ACCEPT: Review skipped due to error.",
                name="manager"
            )],
            "stage_attempts": {**attempts, current: attempt_count + 1},
        }
```

Error Handling:

- Accept by default if review fails
- Prevents blocking pipeline
- Logs error for debugging

Iteration Caps

```python
def specialist_node(state: AnalysisState) -> dict:
    iteration = sum(1 for m in state["messages"]
                    if getattr(m, "name", None) == name) + 1

    if iteration > MAX_SPECIALIST_ITERS:
        print(f"  [{name}] Iteration cap reached, advancing.")
        return {
            "messages": [AIMessage(
                content=f"[{name}] Done (iteration cap). Moving on.",
                name=name,
            )],
        }
```

Why Caps?

- Prevents infinite loops
- Forces progression after reasonable effort
- MAX_SPECIALIST_ITERS = 5, MAX_REWORKS_PER_STAGE = 1

Token Budget Controls

Constants

```
MAX_TOOL_OUTPUT_CHARS = 2000
MAX_AI_MSG_CHARS = 3000
MAX_MESSAGES_WINDOW = 14
MAX_SPECIALIST_ITERS = 5
```

Truncation Function

```
def truncate(text: str, limit: int) -> str:
    return text[:limit] + "\n... [truncated]" if len(text) > limit else text
```

Application Points

1. **Tool output**: Truncate to 2000 chars

   ```
   compressed.append(ToolMessage(
       content=truncate(content, MAX_TOOL_OUTPUT_CHARS),
       ...
   ))
   ```

2. **AI messages**: Truncate to 3000 chars

   ```
   if len(content) > MAX_AI_MSG_CHARS:
       compressed.append(AIMessage(
           content=truncate(content, MAX_AI_MSG_CHARS),
           ...
       ))
   ```

3. **Message window**: Keep only last 14 messages

   ```
   if len(messages) > MAX_MESSAGES_WINDOW + 1:
       messages = [messages[0]] + messages[-(MAX_MESSAGES_WINDOW):]
   ```

4. **Prior summaries**: Keep only last 6

```
summary = "[Prior analysis context]:\n" + \
          "\n".join(prior_summary_parts[-6:])
```

Why These Limits?

- **Tool output**: 2000 chars ≈ 500 tokens (reasonable for tool results)
- **AI messages**: 3000 chars ≈ 750 tokens (agent responses)
- **Message window**: 14 messages ≈ 3500-5000 tokens (conversation context)
- **Prior summaries**: 6 summaries ≈ 1500 tokens (prior stage context)

Total Budget: ~10,000 tokens per agent call (well within typical limits)

Complete Execution Flow

Step-by-Step Pipeline

```
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 1: Initialization                                                   │
│                                                                          │
│ main.py:                                                                 │
│   1. Connect to MCP server                                               │
│   2. Fetch tools via get_mcp_tools()                                     │
│   3. Build graph with build_graph(tools, api_key)                        │
│   4. Invoke graph with user question                                     │
│                                                                          │
│ graph.ainvoke(                                                           │
│   {"messages": [HumanMessage(content=user_input)]},                      │
│   config={"configurable": {"thread_id": label}, "recursion_limit": 100}  │
│ )                                                                        │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 2: Data Loader Stage                                                │
│                                                                          │
│ data_loader_entry:                                                       │
│   → Set current_stage = "data_loader"                                    │
│                                                                          │
│ data_loader:                                                             │
│   → System prompt: "You are the Data Loader..."                          │
│   → Focused context: user request + empty prior context                  │
│   → LLM decides: "I need to load data for Mill 8"                        │
│   → Returns AIMessage with tool_calls                                    │
│                                                                          │
```

```
│ specialist_router → "tools" (has tool_calls)                                  │
│                                                                               │
│ tools:                                                                        │
│   → Execute query_mill_data(mill_number=8, start_date="...", ...)             │
│   → Returns ToolMessage with data summary                                     │
│                                                                               │
│ after_tools → "data_loader" (return to caller)                                │
│                                                                               │
│ data_loader (iteration 2):                                                    │
│   → Receives ToolMessage with loaded data                                     │
│   → No more tool_calls needed                                                 │
│   → Returns AIMessage: "Data loaded successfully"                             │
│                                                                               │
│ specialist_router → "manager_review" (no tool_calls)                          │
│                                                                               │
│ manager_review:                                                               │
│   → Skips review for data_loader                                              │
│   → Returns AIMessage: "ACCEPT: Data loaded successfully"                     │
│                                                                               │
│ manager_router → "analyst_entry" (advance to next)                            │
└───────────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌───────────────────────────────────────────────────────────────────────────────┐
│ STEP 3: Analyst Stage                                                         │
│                                                                               │
│ analyst_entry:                                                                │
│   → Set current_stage = "analyst"                                             │
│                                                                               │
│ analyst:                                                                      │
│   → System prompt: "You are the Data Analyst..."                              │
│   → Focused context: user request + data_loader summary                       │
│   → LLM decides: "I'll analyze the data"                                      │
│   → tool_calls: [list_dfs()]                                                  │
│                                                                               │
│ tools → list_dfs() → returns available dataframes                             │
│ after_tools → "analyst"                                                       │
│                                                                               │
│ analyst (iteration 2):                                                        │
│   → Receives dataframe list                                                   │
│   → tool_calls: [execute_python(code="...")]                                  │
│                                                                               │
│ tools → execute_python() → returns stdout + new_files                         │
│ after_tools → "analyst"                                                       │
│                                                                               │
│ analyst (iteration 3):                                                        │
│   → Receives analysis results                                                 │
│   → No more tool_calls                                                        │
│   → Returns AIMessage: "Analysis complete"                                    │
│                                                                               │
│ specialist_router → "manager_review"                                          │
│                                                                               │
│ manager_review:                                                               │
│   → Evaluates analyst output                                                  │
│   → LLM decision: "ACCEPT: Charts generated with proper formatting"           │
│                                                                               │
│ manager_router → "code_reviewer_entry" (advance)                              │
```

```
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 4: Code Reviewer Stage                                              │
│                                                                          │
│ code_reviewer_entry:                                                     │
│   → Set current_stage = "code_reviewer"                                  │
│                                                                          │
│ code_reviewer:                                                           │
│   → System prompt: "You are the Code Reviewer..."                        │
│   → Focused context: user request + prior summaries                      │
│   → LLM decides: "I'll check the outputs"                                │
│   → tool_calls: [list_output_files()]                                    │
│                                                                          │
│ tools → list_output_files() → returns file list                          │
│ after_tools → "code_reviewer"                                            │
│                                                                          │
│ code_reviewer (iteration 2):                                             │
│   → Receives file list                                                   │
│   → No more tool_calls (validation complete)                             │
│   → Returns AIMessage: "Validation passed"                               │
│                                                                          │
│ specialist_router → "manager_review"                                     │
│                                                                          │
│ manager_review:                                                          │
│   → Evaluates code_reviewer output                                       │
│   → LLM decision: "ACCEPT: Validation passed"                            │
│                                                                          │
│ manager_router → "reporter_entry" (advance)                              │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 5: Reporter Stage                                                   │
│                                                                          │
│ reporter_entry:                                                          │
│   → Set current_stage = "reporter"                                       │
│                                                                          │
│ reporter:                                                                │
│   → System prompt: "You are the Reporter..."                             │
│   → Focused context: user request + prior summaries                      │
│   → LLM decides: "I'll create the report"                                │
│   → tool_calls: [list_output_files()]                                    │
│                                                                          │
│ tools → list_output_files() → returns file list                          │
│ after_tools → "reporter"                                                 │
│                                                                          │
│ reporter (iteration 2):                                                  │
│   → Receives file list                                                   │
│   → Reads through prior messages for statistics                          │
│   → tool_calls: [write_markdown_report(filename="...", content="...")]   │
│                                                                          │
│ tools → write_markdown_report() → saves report                           │
│ after_tools → "reporter"                                                 │
│                                                                          │
│ reporter (iteration 3):                                                  │
```

```
│   → Receives write confirmation                                          │
│   → No more tool_calls                                                   │
│   → Returns AIMessage: "Report written successfully"                     │
│                                                                          │
│ specialist_router → "manager_review"                                     │
│                                                                          │
│ manager_review:                                                          │
│   → Evaluates reporter output                                            │
│   → LLM decision: "ACCEPT: Report complete"                              │
│                                                                          │
│ manager_router → "end" (no more stages)                                  │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 6: Completion                                                       │
│                                                                          │
│ final_state = await graph.ainvoke(...)                                   │
│ final_answer = final_state["messages"][-1].content                       │
│                                                                          │
│ Output: Reporter's final AIMessage content                               │
└──────────────────────────────────────────────────────────────────────────┘
```

Rework Scenario

```
┌──────────────────────────────────────────────────────────────────────────┐
│ SCENARIO: Manager Rejects Analyst Output                                 │
│                                                                          │
│ analyst:                                                                 │
│   → Returns AIMessage: "Analysis complete (minimal)"                     │
│                                                                          │
│ manager_review:                                                          │
│   → Evaluates: "Charts missing, no statistics"                           │
│   → LLM decision: "REWORK: Generate proper charts and statistics"        │
│                                                                          │
│ manager_router → "analyst_entry" (rework same stage)                     │
│                                                                          │
│ analyst_entry:                                                           │
│   → Set current_stage = "analyst" (already set, no change)               │
│                                                                          │
│ analyst (iteration 2, attempt_count = 2):                                │
│   → Focused context includes manager feedback                            │
│   → LLM sees: "REWORK: Generate proper charts and statistics"            │
│   → tool_calls: [execute_python(code="...")]                             │
│   → Generates proper charts and statistics                               │
│   → Returns AIMessage: "Analysis complete with charts"                   │
│                                                                          │
│ manager_review:                                                          │
│   → Evaluates: "Charts present, statistics included"                     │
│   → LLM decision: "ACCEPT: Quality improved"                             │
│                                                                          │
```

```
│ manager_router → "code_reviewer_entry" (advance)                    │
└──────────────────────────────────────────────────────────────────────┘
```

Extension Guide

Adding a New Specialist

Step 1: Define System Prompt

```
NEW_SPECIALIST_PROMPT = f"""{DOMAIN_CONTEXT}

You are the [Specialist Name]. Your job is to [describe role].

[Specific instructions for this specialist]

[Tool usage rules]
[Output requirements]
"""
```

Step 2: Add to STAGES

```
STAGES = ["data_loader", "analyst", "code_reviewer", "reporter", "new_specialist"]
```

Step 3: Define Tool Set

```
TOOL_SETS = {
    "data_loader": ["query_mill_data", "query_combined_data", "get_db_schema"],
    "analyst": ["execute_python", "list_output_files"],
    "code_reviewer": ["execute_python", "list_output_files"],
    "reporter": ["list_output_files", "write_markdown_report"],
    "new_specialist": ["tool1", "tool2"],  # Add tools for new specialist
}
```

Step 4: Add to PROMPTS

```
PROMPTS = {
    "data_loader": DATA_LOADER_PROMPT,
```

```
    "analyst": ANALYST_PROMPT,
    "code_reviewer": CODE_REVIEWER_PROMPT,
    "reporter": REPORTER_PROMPT,
    "new_specialist": NEW_SPECIALIST_PROMPT,  # Add prompt
}
```

Step 5: Update Manager Review (if needed)

```
def manager_review_node(state: AnalysisState) -> dict:
    current = state.get("current_stage", "data_loader")

    # Skip review for certain stages if needed
    if current in ["data_loader", "new_specialist"]:
        return {...}

    # ... rest of review logic
```

Adding New Routing Logic

Example: Conditional Specialist Selection

```
def specialist_router(state: AnalysisState) -> str:
    last = state["messages"][-1]
    current = state.get("current_stage", "data_loader")

    if hasattr(last, "tool_calls") and last.tool_calls:
        return "tools"

    # Custom routing based on stage
    if current == "analyst" and some_condition:
        return "special_reviewer"  # Go to special reviewer

    return "manager_review"
```

Modifying Context Building

Example: Add More Context for Specific Stage

```
def build_focused_context(all_msgs: list[BaseMessage], stage_name: str) ->
list[BaseMessage]:
    # ... existing logic ...
```

```
    # Add custom context for specific stage
    if stage_name == "analyst":
        # Include more prior stage details
        prior_summary_parts = prior_summary_parts[-10:]  # Keep 10 instead of 6

    # ... rest of logic ...
```

Adding Error Recovery

Example: Retry Failed Tool Calls

```
async def tool_node(state: AnalysisState) -> dict:
    last_message = state["messages"][-1]
    results = []

    for tc in last_message.tool_calls:
        tool = tools_by_name.get(tc["name"])
        if tool is None:
            results.append(ToolMessage(
                content=f"Error: unknown tool '{tc['name']}'",
                tool_call_id=tc["id"],
                name=tc["name"],
            ))
            continue

        # Retry logic
        max_retries = 3
        for attempt in range(max_retries):
            try:
                output = await tool.ainvoke(tc["args"])
                results.append(ToolMessage(content=str(output), ...))
                break
            except Exception as e:
                if attempt == max_retries - 1:
                    results.append(ToolMessage(
                        content=f"Error: {e} (after {max_retries} retries)",
                        tool_call_id=tc["id"],
                        name=tc["name"],
                    ))
                else:
                    await asyncio.sleep(1)  # Wait before retry

    return {"messages": results}
```

Summary

Graph V2 implements a **deterministic multi-agent workflow** with:

Key Features:

- **Specialist agents**: Each with specific role and tools
- **Manager QA review**: Quality gate between stages
- **Tool execution**: MCP tools for data loading, analysis, reporting
- **Focused context**: Token-efficient message handling
- **Rework capability**: Manager can request improvements
- **Error handling**: Graceful failure at multiple levels

Design Patterns:

- **State management**: Shared state with message history
- **Conditional routing**: Dynamic flow based on agent decisions
- **Context building**: Focused, relevant information per agent
- **Token budgeting**: Prevents exceeding LLM limits

Benefits:

- **Modularity**: Easy to add new specialists
- **Testability**: Each agent can be tested independently
- **Scalability**: Supports complex multi-stage workflows
- **Reliability**: Deterministic flow with quality gates

Next Steps:

- Explore individual specialist implementations
- Review system prompts for role definitions
- Test with different analysis requests
- Extend with custom specialists and tools